

|                                                                                                                                 |                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <br><b>Arquitectura con Patrones de Diseño</b> | Departamento: Ciencias de la Computación                                                                                              |
|                                                                                                                                 | Carrera: Ingeniería en Software                                                                                                       |
|                                                                                                                                 | Asignatura: Análisis y Diseño de Software<br>NRC: 30723                                                                               |
|                                                                                                                                 | Apellidos y nombres del estudiante:<br><br>Cedeño Cuenca Andrés Isaías<br>Santi Cruz Jeancarlo Javier<br>Tufiño Ortiz Erick Alejandro |

## Índice de Contenidos

|                                                                                                                |          |
|----------------------------------------------------------------------------------------------------------------|----------|
| <b>1. U2T1. Diseño e implementación de la arquitectura de tres capas para el CRUD del estudiante..</b>         | <b>3</b> |
| 1.1. Antecedentes.....                                                                                         | 3        |
| 1.2. Objetivo.....                                                                                             | 3        |
| 1.3. Desarrollo.....                                                                                           | 3        |
| 1.3.1. Proyecto Base Organizado en tres capas.....                                                             | 3        |
| 1.3.2. Diagrama de arquitectura aplicado al CRUD del estudiante.....                                           | 4        |
| 1.3.3. Tabla de responsabilidades por capa.....                                                                | 5        |
| 1.3.4. Matriz de trazabilidad entre requisitos del CRUD y componentes implementados.....                       | 5        |
| 1.3.5. Capturas y Pruebas de ejecución del registro, consulta, actualización y eliminación de estudiantes..... | 6        |
| 1.4. Conclusión.....                                                                                           | 6        |
| 1.5. Recomendación.....                                                                                        | 6        |
| <b>2. U2T2. Aplicación de Adapter y Decorator en el CRUD del estudiante.....</b>                               | <b>7</b> |
| 2.1. Antecedentes.....                                                                                         | 7        |
| 2.2. Objetivo.....                                                                                             | 7        |
| 2.3. Desarrollo.....                                                                                           | 7        |
| 2.3.1. Implementación funcional del Adaptes en el CRUD del estudiante.....                                     | 7        |
| 2.3.2. Implementación funcional del Decorator en el servicio o componente seleccionado.....                    | 7        |
| 2.3.3. Diagrama actualizado con los patrones aplicados.....                                                    | 7        |
| 2.3.4. Justificación técnica del uso de cada patrón.....                                                       | 7        |
| 2.3.5. Pruebas y Capturas de la Ejecución.....                                                                 | 7        |
| 2.4. Conclusión.....                                                                                           | 7        |
| 2.5. Recomendación.....                                                                                        | 7        |
| <b>3. U2T3. Aplicación de Observer y Strategy en el CRUD del estudiante.....</b>                               | <b>7</b> |
| 3.1. Antecedentes.....                                                                                         | 7        |
| 3.2. Objetivo.....                                                                                             | 7        |
| 3.3. Desarrollo.....                                                                                           | 7        |
| 3.3.1. Implementación funcional del patrón Observer.....                                                       | 7        |
| 3.3.2. Implementación funcional del patrón Strategy.....                                                       | 7        |
| 3.3.3. Diagrama actualizado de los patrones de comportamiento.....                                             | 7        |

|                                                                                         |   |
|-----------------------------------------------------------------------------------------|---|
| 3.3.4. Matriz de trazabilidad entre requisitos, eventos, estrategias y componentes..... | 7 |
| 3.3.5. Evidencias de ejecución y reflexión técnica.....                                 | 7 |
| 3.4. Conclusión.....                                                                    | 7 |
| 3.5. Recomendación.....                                                                 | 7 |

## 1. U2T1. Diseño e implementación de la arquitectura de tres capas para el CRUD del estudiante

### 1.1. Antecedentes

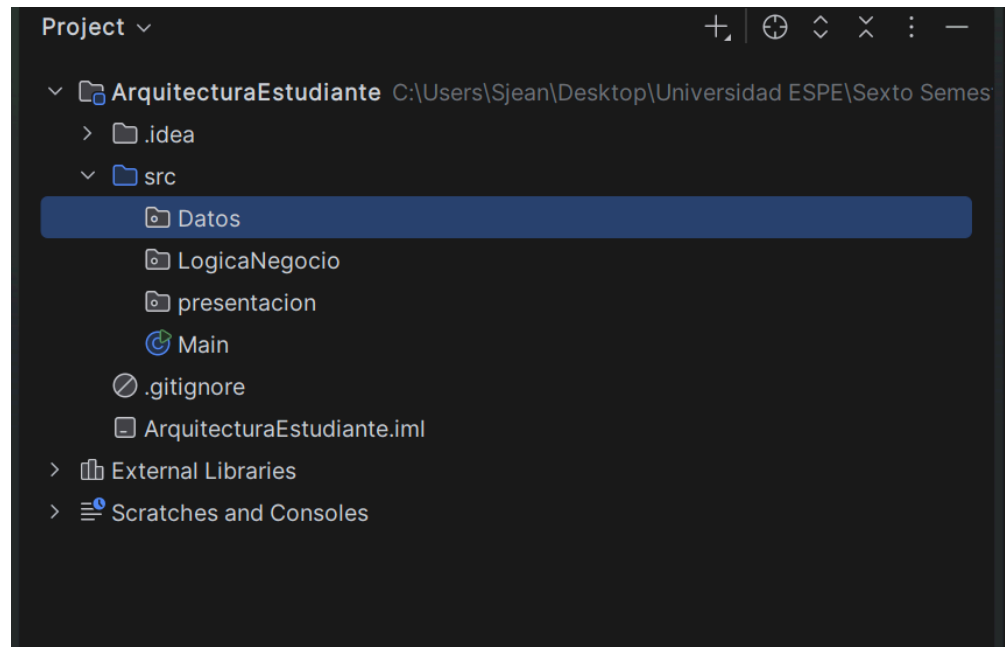
En esta asignatura de Análisis y Diseño y en el desarrollo de software en general la Arquitectura de software constituye un elemento fundamental en el desarrollo de sistemas informáticos, ya que nos permite organizar los distintos componentes que tiene una aplicación y definir claramente sus responsabilidades. Una de las arquitecturas más utilizadas en aplicaciones empresariales es la arquitectura de las tres capas, la cual divide el sistema en capa de Presentación, capa Lógica\_Negocio y capa de Datos, favoreciendo así la separación de responsabilidades, la mantenibilidad y la escalabilidad.

### 1.2. Objetivo

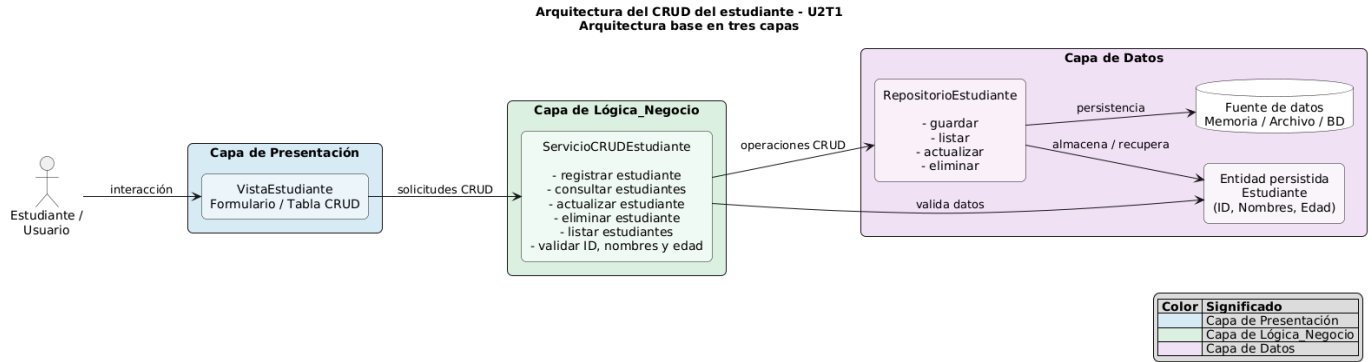
El objetivo de esta tarea es que se diseñe y se implemente una arquitectura de software en tres capas para el CRUD del estudiante, separando adecuadamente las responsabilidades de presentación, lógica de negocio y datos..

### 1.3. Desarrollo

#### 1.3.1. Proyecto Base Organizado en tres capas



### 1.3.2. Diagrama de arquitectura aplicado al CRUD del estudiante



#### Código en PlantUml

```

@startuml U2T1_Arquitectura_Base
title Arquitectura del CRUD del estudiante - U2T1\nArquitectura base en tres capas

left to right direction
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam rectangleRoundCorner 12
skinparam componentStyle rectangle
skinparam packageStyle rectangle

actor "Estudiante /\nUsuario" as Usuario

rectangle "Capa de Presentación" as CAPA_PRESENTACION #D9EAF7 {
    rectangle "VistaEstudiante\nFormulario / Tabla CRUD" as Vista #EAF4FB
}

rectangle "Capa de Lógica_Negocio" as CAPA_NEGOCIO #DFF2E1 {
    rectangle "ServicioCRUDEstudiante\n\n- registrar estudiante\n- consultar\nestudiantes\n- actualizar estudiante\n- eliminar estudiante\n- listar estudiantes\n- validar ID, nombres y edad" as Servicio #F3FBF4
}

rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {
    rectangle "RepositorioEstudiante\n\n- guardar\n- listar\n- actualizar\n- eliminar" as Repositorio #FAF1FC

    database "Fuente de datos\nMemoria / Archivo / BD" as FuenteDatos #FFFFFF

    rectangle "Entidad persistida\nEstudiante\n(ID, Nombres, Edad)" as Entidad #FCF7FD
}

Usuario --> Vista : interacción
Vista --> Servicio : solicitudes CRUD
Servicio --> Repositorio : operaciones CRUD
Repositorio -- persistencia --> FuenteDatos
Repositorio -- almacena / recupera --> Entidad
Repositorio -- valida datos --> Entidad
  
```

Usuario --> Vista : interacción  
 Vista --> Servicio : solicitudes CRUD  
 Servicio --> Repositorio : operaciones CRUD  
 Repositorio --> FuenteDatos : persistencia  
 Repositorio --> Entidad : almacena / recupera  
 Repositorio --> Entidad : valida datos

Servicio --> Entidad : valida datos

legend right

|= Color |= Significado |

|<#D9EAF7>| Capa de Presentación |

|<#DFF2E1>| Capa de Lógica\_Negocio |

|<#F3E0F7>| Capa de Datos |

endlegend

@enduml

### 1.3.3. Tabla de responsabilidades por capa

| Capa           | Componente             | Responsabilidad                                                                                |
|----------------|------------------------|------------------------------------------------------------------------------------------------|
| Presentación   | VistaEstudiante        | Gestionar la interacción con el usuario mediante el menú y las opciones CRUD                   |
| Lógica_Negocio | ServicioCRUDEstudiante | Aplicar validaciones, controlar el flujo de las operaciones y coordinar el acceso a los datos. |
| Datos          | RepositorioEstudiante  | Administrar el almacenamiento, consulta, actualización y eliminación de estudiantes.           |
| Datos          | Estudiante             | Representar la entidad principal del sistema mediante los atributos ID, Nombres y Edad         |

### 1.3.4. Matriz de trazabilidad entre requisitos del CRUD y componentes implementados

| Código  | Requisito Funcional   | Operación | Componente Responsable                                                 |
|---------|-----------------------|-----------|------------------------------------------------------------------------|
| RF - 01 | Registrar Estudiante  | Create    | VistaEstudiante →<br>ServicioCRUDEstudiante →<br>RepositorioEstudiante |
| RF - 02 | Consultar Estudiante  | Read      | VistaEstudiante →<br>ServicioCRUDEstudiante →<br>RepositorioEstudiante |
| RF - 03 | Actualizar Estudiante | Update    | VistaEstudiante →<br>ServicioCRUDEstudiante →<br>RepositorioEstudiante |
| RF - 04 | Eliminar Estudiante   | Delete    | VistaEstudiante →<br>ServicioCRUDEstudiante →                          |

|         |                               |              |                                      |
|---------|-------------------------------|--------------|--------------------------------------|
|         |                               |              | RepositorioEstudiante                |
| RF - 05 | Validar datos del estudiante  | Validación   | ServicioCRUDEstudiante               |
| RF - 06 | Mantener separación por capas | Arquitectura | Presentación, Lógica_Negocio y Datos |

### 1.3.5. Capturas y Pruebas de ejecución del registro, consulta, actualización y eliminación de estudiantes.

```

Run Main x
5. Eliminar estudiante
0. Salir
=====
Seleccione una opción: 1
--- Registrar estudiante ---
Ingrese ID: 1
Ingrese nombres: Jeancarlo Santi
Ingrese edad: 21
Estudiante registrado correctamente.

4. Actualizar estudiante
5. Eliminar estudiante
0. Salir
=====
Seleccione una opción: 3
--- Consultar estudiante por ID ---
Ingrese ID a consultar: 1
Estudiante encontrado:
ID: 1 | Nombres: Jeancarlo Santi | Edad: 21

5. Eliminar estudiante
0. Salir
=====
Seleccione una opción: 4
--- Actualizar estudiante ---
Ingrese ID del estudiante a actualizar: 1
Ingrese nuevos nombres: Erick Tufino
Ingrese nueva edad: 23
Estudiante actualizado correctamente.

```

### 1.4. Conclusión

Como grupo creemos que la implementación del CRUD estudiante es un ejemplo perfecto que nos permitió aplicar correctamente una arquitectura de tres capas, separando las responsabilidades de presentación, lógica de negocio y datos. Esta organización nos ayuda a mejorar la claridad del diseño, nos facilita el mantenimiento del sistema y además nos garantiza que las validaciones se ejecuten en la capa correspondiente. Además, al tener bien definida la arquitectura desde el inicio tenemos una base sólida para posteriormente seguir implementando patrones de diseño con mayor comodidad sin preocuparnos de romper la funcionalidad del programa.

### 1.5. Recomendación

Recomendamos mantener la separación de responsabilidades entre capas durante futuras ampliaciones del proyecto, incorporando nuevos componentes sin afectar la estructura base. Asimismo recomendamos, documentar adecuadamente cada modificación para facilitar la trazabilidad y la aplicación posterior de patrones de diseño

## 2. U2T2. Aplicación de Adapter y Decorator en el CRUD del estudiante

### 2.1. Antecedentes

La arquitectura de tres capas implementada en la sección previa (U2T1) proporciona una separación de responsabilidades clara y estructurada entre los datos, la lógica del negocio y la interfaz de usuario. En este escenario la introducción de los patrones de diseño estructurales Adapter y Decorator ofrece una solución. El primero actúa como un puente de transformación ante la incompatibilidad de interfaces externas, mientras que el segundo permite adjuntar responsabilidades adicionales de manera dinámica, aislando el comportamiento transaccional del núcleo de la aplicación.

### 2.2. Objetivo

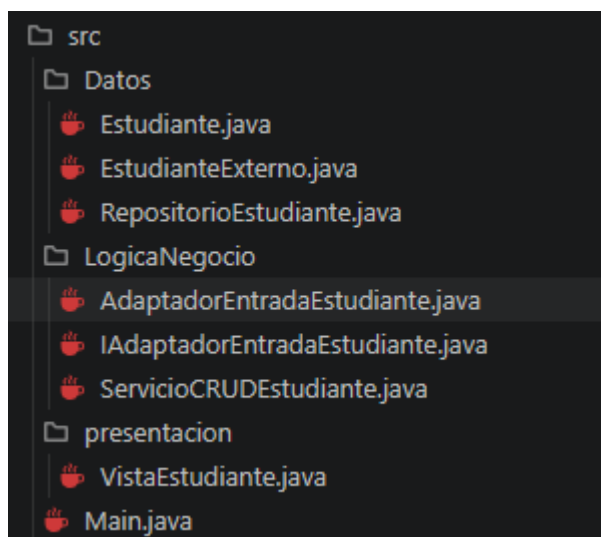
Diseñar e implementar los patrones de diseño estructurales Adapter y Decorator sobre la arquitectura de tres capas desarrollada para el CRUD del estudiante en U2T1.

### 2.3. Desarrollo

#### 2.3.1. Implementación funcional del Adapter en el CRUD del estudiante

El patrón Adapter se aplica para resolver un problema de incompatibilidad de interfaces o estructuras de datos.

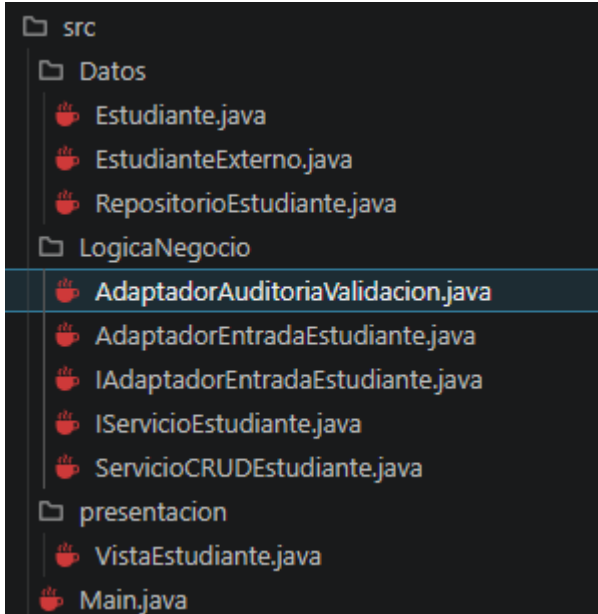
Se modela un escenario donde un sistema externo proporciona información de estudiantes empleando la nomenclatura código, nombreCompleto y años. Mediante la interfaz **IAdaptadorEntradaEstudiante** y su implementación concreta **AdaptadorEntradaEstudiante**, se realiza un mapeo y conversión segura hacia la entidad de dominio interna Estudiante (id, nombres, edad) requerida por el sistema.



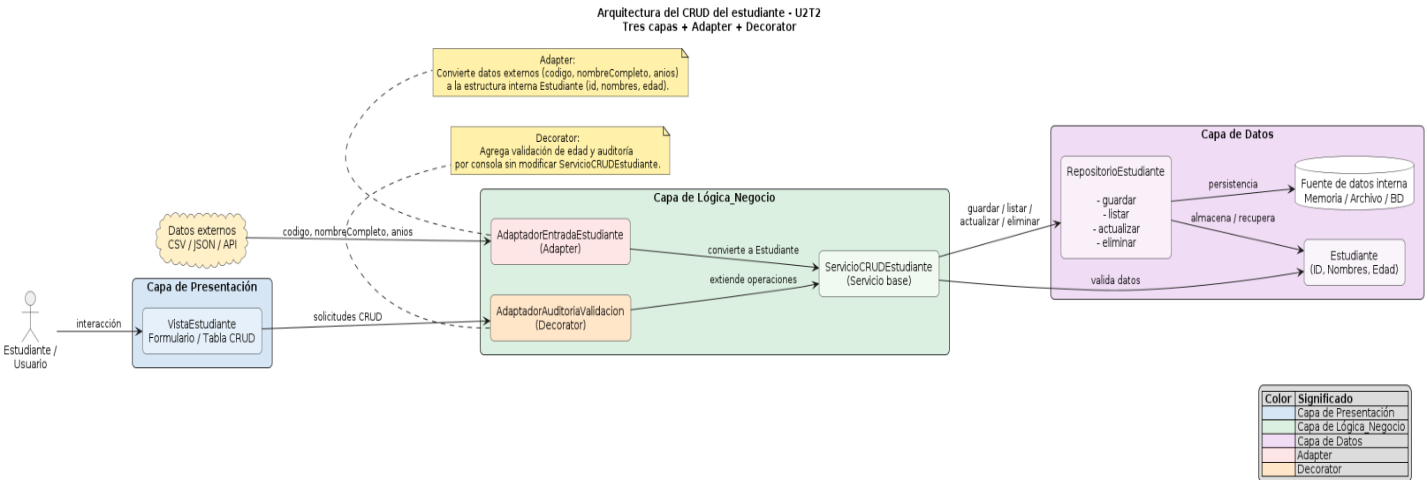
### Implementación funcional del Decorator en el servicio o componente seleccionado

El patrón Decorator se introduce para inyectar dinámicamente funcionalidades adicionales a la capa de negocio.

Para lograr esto, se extrae la interfaz abstracta **IServicioEstudiante** que normaliza los servicios del negocio. La clase original **ServicioCRUDEstudiante** se suscribe a este contrato. El decorador concreto **AdaptadorAuditoriaValidacion** implementa la misma interfaz y envuelve al servicio original. De esta forma, intercepta de manera transparente las peticiones para realizar una validación preventiva del estado del objeto y ejecutar trazas de auditoría en consola antes y después de interactuar con el almacenamiento persistente.



### 2.3.3. Diagrama actualizado con los patrones aplicados





## Código en PlantUml

```
@startuml U2T2_Arquitectura_Structural
title Arquitectura del CRUD del estudiante - U2T2\nTres capas + Adapter + Decorator

left to right direction
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam rectangleRoundCorner 12
skinparam componentStyle rectangle
skinparam packageStyle rectangle

actor "Estudiante \nUsuario" as Usuario

cloud "Datos externos\nCSV / JSON / API" as Externo #FFF2CC

rectangle "Capa de Presentación" as CAPA_PRESENTACION #D9EAF7 {
    rectangle "VistaEstudiante\nFormulario / Tabla CRUD" as Vista #EAF4FB
}

rectangle "Capa de Lógica_Negocio" as CAPA_NEGOCIO #DFF2E1 {
    rectangle "AdaptadorAuditoriaValidacion\n(Decorator)" as Decorator #FFE6CC
    rectangle "AdaptadorEntradaEstudiante\n(Adapter)" as Adapter #FFE6E6
    rectangle "ServicioCRUDEstudiante\n(Servicio base)" as Servicio #F3FBF4
}

rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {
    rectangle "RepositorioEstudiante\n\n guardar\n- listar\n- actualizar\n- eliminar" as Repositorio #FAF1FC
    database "Fuente de datos interna\nMemoria / Archivo / BD" as FuenteDatos #FFFFFF
    rectangle "Estudiante\n(ID, Nombres, Edad)" as Entidad #FCF7FD
}

Usuario --> Vista : interacción
Vista --> Decorator : solicitudes CRUD

Decorator --> Servicio : extiende operaciones
Adapter --> Servicio : convierte a Estudiante
Externo --> Adapter : codigo, nombreCompleto, anios

Servicio --> Repositorio : guardar / listar \nactualizar / eliminar
Repositorio --> FuenteDatos : persistencia
Repositorio --> Entidad : almacena / recupera
Servicio --> Entidad : valida datos

note bottom of Decorator #FFF2AA
Decorator:
Agrega validación de edad y auditoría
por consola sin modificar ServicioCRUDEstudiante.
end note
```

```

note bottom of Adapter #FFF2AA
Adapter:
Convierte datos externos (codigo, nombreCompleto, anios)
a la estructura interna Estudiante (id, nombres, edad).
end note

```

```

legend right
|= Color |= Significado |
|<#D9EAF7>| Capa de Presentación |
|<#DFF2E1>| Capa de Lógica_Negocio |
|<#F3E0F7>| Capa de Datos |
|<#FFE6E6>| Adapter |
|<#FFE6CC>| Decorator |
endlegend

```

```
@enduml
```

#### 2.3.4. Justificación técnica del uso de cada patrón

- Justificación de Adapter (AdaptadorEntradaEstudiante):** El uso de este patrón resolvió el problema de incompatibilidad que ocurre cuando los datos del estudiante provenían de una fuente externa estructurada con los atributos codigo, nombreCompleto y anios a través de la clase EstudianteExterno. Nuestra capa de datos interna y el repositorio están diseñados de forma estricta para operar únicamente con la entidad nativa Estudiante bajo los campos id, nombres y edad. En lugar de modificar la entidad base o duplicar métodos de inserción en el servicio, la implementación del adaptador resolvió el problema al centralizar la traducción de tipos (como el parseo de String a int para el identificador). Esto justificó su uso al permitir la interoperabilidad con EstudianteExterno sin romper el flujo transaccional nativo ni alterar el código que ya funcionaba.
- Justificación de Decorator (AdaptadorAuditoriaValidacion):** Su implementación resolvió el problema de acoplamiento que significaba tener que añadir requerimientos de infraestructura (como la bitácora de auditoría por consola) y validaciones perimetrales restrictivas (como bloquear el registro si la edad ingresada es menor o igual a cero) dentro del flujo puro del servicio base ServicioCRUDEstudiante. Al aplicar el decorador, resolvimos la necesidad de extender las capacidades del negocio sin alterar ni una sola línea de código de la clase base heredada de la tarea anterior. Esto justifica técnicamente su adopción porque el control transaccional adicional se ejecuta de forma periférica e invisible para la capa de presentación, garantizando que cada componente mantenga una cohesión alta y protegiendo el sistema de la rigidez que provocaría el uso de la herencia tradicional.

### 2.3.5. Pruebas y Capturas de la Ejecución

Comportamiento anterior a la implementación de los patrones de diseño

```
=====
CRUD DEL ESTUDIANTE - ARQUITECTURA INTEGRADA U2T2
Estado del Decorador: DESACTIVADO (Flujo Base U2T1)
=====
1. Registrar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
-----
6. [DECORATOR] Activar/Desactivar Auditoria y Validación
7. [ADAPTER] Cargar datos de EstudianteExterno
0. Salir
=====
Seleccione una opción: 1
```

```
Seleccione una opción: 1

--- Registrar estudiante ---
Ingreso ID: 1
Ingreso nombres: Andres Cedeño
Ingreso edad: 21
Estudiante registrado correctamente.
```

Comportamiento con el patrón de diseño Decorator

```
=====
CRUD DEL ESTUDIANTE - ARQUITECTURA INTEGRADA U2T2
Estado del Decorador: DESACTIVADO (Flujo Base U2T1)
=====
1. Registrar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
-----
6. [DECORATOR] Activar/Desactivar Auditoria y Validación
7. [ADAPTER] Cargar datos de EstudianteExterno
0. Salir
=====
Seleccione una opción: 6
```

```
[SISTEMA] Decorador Activado con éxito. Se inyectaron Logs de Auditoria y Validación.
```

```
=====
CRUD DEL ESTUDIANTE - ARQUITECTURA INTEGRADA U2T2
Estado del Decorador: ACTIVADO (Decorator)
=====
1. Registrar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
-----
6. [DECORATOR] Activar/Desactivar Auditoria y Validación
7. [ADAPTER] Cargar datos de EstudianteExterno
0. Salir
=====
Seleccione una opción: 1
```

```
=====
Seleccione una opción: 1

--- Registrar estudiante ---
Ingreso ID: 2
Ingreso nombres: Erick Tufiño
Ingreso edad: 24

=====
[AUDITORIA DECORATOR] -> Ejecutando operación: Registrar
[AUDITORIA DECORATOR] -> Datos evaluados: ID: 2 | Nombre: Erick Tufiño | Edad: 24
[AUDITORIA DECORATOR] -> Resultado del registro base: Estudiante registrado correctamente.
=====
Estudiante registrado correctamente.
```

```
=====
Seleccione una opción: 1

--- Registrar estudiante ---
Ingreso ID: 3
Ingreso nombres: Jeancarlo Santi
Ingreso edad: 0
Error de Validación [DECORATOR]: La edad debe ser mayor a cero.
```

## Comportamiento con el patrón de diseño Adapter

```
=====
CRUD DEL ESTUDIANTE - ARQUITECTURA INTEGRADA U2T2
Estado del Decorador: ACTIVADO (Decorator)
=====
1. Registrar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
-----
6. [DECORATOR] Activar/Desactivar Auditoría y Validación
7. [ADAPTER] Cargar datos de EstudianteExterno
8. Salir
=====
Seleccione una opción: 7

----- Simulación de Entrada Externa (CSV/JSON/API) -----
Se simulará la recepción del objeto incompatible 'EstudianteExterno'.
Ingrese campo 'codigo' (String): 202650
Ingrese campo 'nombreCompleto' (String): Jeancarlo Santi
Ingrese campo 'anios' (int): 22

[PROCESO] Enviando datos al AdaptadorEntradaEstudiante...
[ADAPTER SUCCESS] Objeto adaptado al dominio interno correctamente.
Datos homologados -> ID: 202650 | Nombres: Jeancarlo Santi | Edad: 22

[PROCESO] Enviando entidad adaptada al flujo de registro del Servicio actual...

[AUDITORÍA DECORATOR] -> Ejecutando operación: Registrar
[AUDITORÍA DECORATOR] -> Datos evaluados: ID: 202650 | Nombre: Jeancarlo Santi
| Edad: 22
[AUDITORÍA DECORATOR] -> Resultado del registro base: Estudiante registrado cor
rectamente.

=====
Estudiante registrado correctamente.

=====
Seleccione una opción: 7

----- Simulación de Entrada Externa (CSV/JSON/API) -----
Se simulará la recepción del objeto incompatible 'EstudianteExterno'.
Ingrese campo 'codigo' (String): 202650
Ingrese campo 'nombreCompleto' (String): Jeancarlo Santi
Ingrese campo 'anios' (int): 22

[PROCESO] Enviando datos al AdaptadorEntradaEstudiante...
[ADAPTER SUCCESS] Objeto adaptado al dominio interno correctamente.
Datos homologados -> ID: 202650 | Nombres: Jeancarlo Santi | Edad: 22

[PROCESO] Enviando entidad adaptada al flujo de registro del Servicio actual...

[AUDITORÍA DECORATOR] -> Ejecutando operación: Registrar
[AUDITORÍA DECORATOR] -> Datos evaluados: ID: 202650 | Nombre: Jeancarlo Santi
| Edad: 22
[AUDITORÍA DECORATOR] -> Resultado del registro base: Estudiante registrado cor
rectamente.

=====
Estudiante registrado correctamente.

=====
Seleccione una opción: 2

--- Lista de estudiantes ---
ID: 1 | Nombres: Andres Cedeño | Edad: 21
ID: 2 | Nombres: Erick Tufiño | Edad: 24
ID: 202650 | Nombres: Jeancarlo Santi | Edad: 22
=====
```

### 2.4. Conclusión

Como grupo creemos que la integración de los patrones estructurales representa una mejora sobre el CRUD de estudiante. Pensamos que el uso de Adapter resolvió de forma limpia la incompatibilidad de formatos sin perturbar el repositorio de datos, mientras que con Decorator logramos incorporar trazas de control e infraestructura sin alterar los flujos operacionales base. Consideramos que esta evolución demuestra cómo los patrones mitigan la rigidez arquitectónica, garantizando que el sistema sea modular, tolerante a los cambios externos y considerablemente más fácil de mantener.

### 2.5. Recomendación

Recomendamos firmemente que para las futuras ampliaciones del sistema, evitemos instanciar las dependencias encadenadas directamente dentro del formulario de la interfaz de usuario de VistaEstudiante. Asimismo sugerimos la adopción de una clase de fábrica (Factory) o un mecanismo de inyección de dependencias para delegar la construcción de objetos compuestos como `new AdaptadorAuditoriaValidacion(new ServicioCRUDEstudiante())`. Estimamos que esto preservará la pureza de la separación por capas conseguida, asegurando que la presentación no conozca la lógica de envoltura de los decoradores o los adaptadores del negocio.

### 3. U2T3. Aplicación de Observer y Strategy en el CRUD del estudiante

#### 3.1. Antecedentes

Una vez consolidada la arquitectura de tres capas y resueltos los problemas de incompatibilidad de interfaces y extensión de responsabilidades mediante patrones estructurales, el sistema CRUD del estudiante requería mecanismos eficientes para gestionar su comportamiento dinámico. En sistemas altamente cohesivos, la lógica de negocio no debe acoplarse rígidamente a los algoritmos que utiliza ni debe ser la encargada de invocar explícitamente a los módulos de notificación cada vez que ocurre un cambio de estado. Para superar estas limitaciones y evitar el uso desmedido de sentencias condicionales, se introduce el diseño basado en patrones de comportamiento.

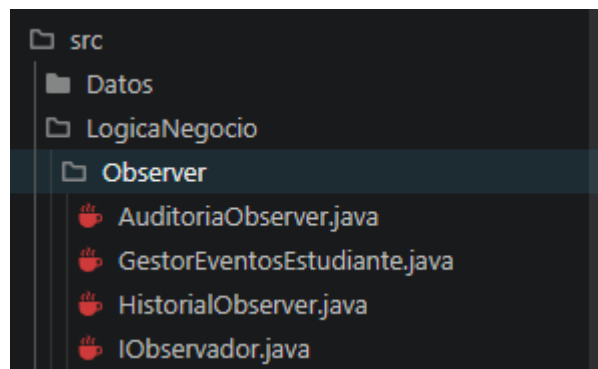
#### 3.2. Objetivo

Aplicar los patrones de comportamiento Observer y Strategy sobre el CRUD del estudiante para gestionar eventos transaccionales del sistema y permitir la selección dinámica de comportamientos.

#### 3.3. Desarrollo

##### 3.3.1. Implementación funcional del patrón Observer

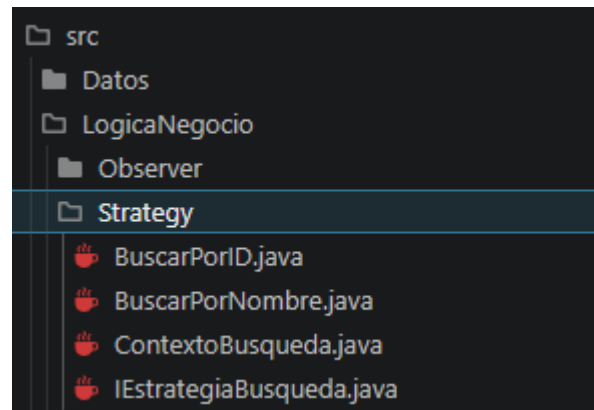
El patrón Observer se implementó para aislar la lógica transaccional de los mecanismos de alerta y registro. Se definió el servicio principal como el "Sujeto" o publicador, el cual administra una lista de suscriptores dinámicos. Cuando se completa exitosamente el registro, la actualización o la eliminación de un estudiante en la base de datos, el servicio emite una notificación estandarizada. Por su parte, se implementaron dos observadores concretos: uno encargado de emitir alertas de auditoría en vivo por consola y otro dedicado a almacenar silenciosamente un historial de todos los eventos ocurridos en la sesión.



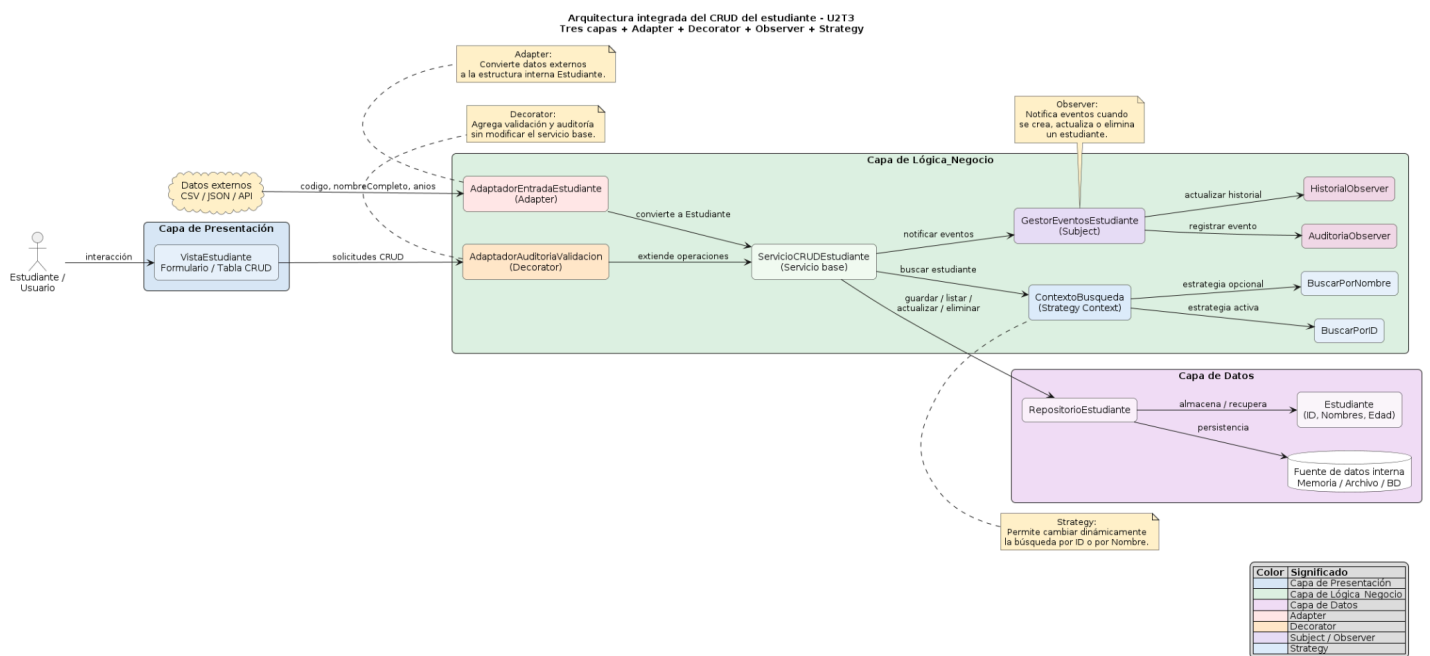
##### 3.3.2. Implementación funcional del patrón Strategy

El patrón Strategy se integró para abstraer la lógica de filtrado y búsqueda de estudiantes. En lugar de tener un único método monolítico saturado de condicionales (ej. `if (buscarPorId) {...} else if (buscarPorNombre) {...}`), se definió una interfaz común para la estrategia de búsqueda. A partir de esta, se crearon dos algoritmos intercambiables: la búsqueda exacta por identificador único (ID) y la búsqueda por aproximación de cadena de texto (Nombre). Un componente de contexto se encarga de recibir la estrategia seleccionada por el usuario en

tiempo de ejecución y ejecutarla sobre el repositorio de datos sin que la capa de presentación deba conocer los detalles internos de implementación.



### 3.3.3. Diagrama actualizado de los patrones de comportamiento



```
@startuml U2T3_Arquitectura_Observer_Strategy
title Arquitectura integrada del CRUD del estudiante - U2T3\nTres capas + Adapter +
Decorator + Observer + Strategy

left to right direction
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam rectangleRoundCorner 12
skinparam componentStyle rectangle
skinparam packageStyle rectangle

actor "Estudiante \nUsuario" as Usuario

cloud "Datos externos\nCSV / JSON / API" as Externo #FFF2CC

rectangle "Capa de Presentación" as CAPA_PRESENTACION #D9EAF7 {
    rectangle "VistaEstudiante\nFormulario / Tabla CRUD" as Vista #EAF4FB
}

rectangle "Capa de Lógica_Negocio" as CAPA_NEGOCIO #DFF2E1 {

    rectangle "AdaptadorAuditoriaValidacion\n(Decorator)" as Decorator #FFE6CC
    rectangle "AdaptadorEntradaEstudiante\n(Adapter)" as Adapter #FFE6E6
    rectangle "ServicioCRUDEstudiante\n(Servicio base)" as Servicio #F3FBF4

    rectangle "GestorEventosEstudiante\n(Subject)" as Subject #EADCF8
    rectangle "AuditoriaObserver" as ObsAuditoria #F4D9E7
    rectangle "HistorialObserver" as ObsHistorial #F4D9E7

    rectangle "ContextoBusqueda\n(Strategy Context)" as CtxBusqueda #DCEBFA
    rectangle "BuscarPorID" as EstrategiaID #E8F4FD
    rectangle "BuscarPorNombre" as EstrategiaNombre #E8F4FD
}

rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {
    rectangle "RepositorioEstudiante" as Repositorio #FAF1FC
    database "Fuente de datos interna\nMemoria / Archivo / BD" as FuenteDatos
    #FFFFFF
    rectangle "Estudiante\n(ID, Nombres, Edad)" as Entidad #FCF7FD
}

Usuario --> Vista : interacción
Vista --> Decorator : solicitudes CRUD

Decorator --> Servicio : extiende operaciones
Adapter --> Servicio : convierte a Estudiante
Externo --> Adapter : codigo, nombreCompleto, anios

Servicio --> CtxBusqueda : buscar estudiante
CtxBusqueda --> EstrategiaID : estrategia activa
```



CtxBusqueda --> EstrategiaNombre : estrategia opcional

Servicio --> Subject : notificar eventos

Subject --> ObsAuditoria : registrar evento

Subject --> ObsHistorial : actualizar historial

Servicio --> Repositorio : guardar / listar / actualizar / eliminar

Repositorio --> FuenteDatos : persistencia

Repositorio --> Entidad : almacena / recupera

note top of CtxBusqueda #FFF2CC

Strategy:

Permite cambiar dinámicamente  
la búsqueda por ID o por Nombre.  
end note

note top of Subject #FFF2CC

Observer:

Notifica eventos cuando  
se crea, actualiza o elimina  
un estudiante.  
end note

note bottom of Decorator #FFF2CC

Decorator:

Agrega validación y auditoría  
sin modificar el servicio base.  
end note

note bottom of Adapter #FFF2CC

Adapter:

Convierte datos externos  
a la estructura interna Estudiante.  
end note

legend right

|= Color |= Significado |

<#D9EAF7>| Capa de Presentación |

<#DFF2E1>| Capa de Lógica\_Negocio |

<#F3E0F7>| Capa de Datos |

<#FFE6E6>| Adapter |

<#FFE6CC>| Decorator |

<#EADCF8>| Subject / Observer |

<#DCEBFA>| Strategy |

endlegend

@enduml



### 3.3.4. Matriz de trazabilidad entre requisitos, eventos, estrategias y componentes

| Requisito / Entrada                       | U2T1 Arquitectura                                                                      | U2T2 Patrones Estructurales                                                              | U2T3 Patrones de Comportamiento                                                                                                | Evidencia Final                      |
|-------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| <b>RF-01 al RF-04: Operaciones CRUD</b>   | Se implementan operaciones base de la entidad estudiante en la capa de Datos y Lógica. | Se adaptan fuentes externas incompatibles.                                               | Se agregan notificaciones de evento tras cada éxito transaccional.                                                             | Pruebas de ejecución en terminal.    |
| <b>RNF: Mantenibilidad y Trazabilidad</b> | Se separan las responsabilidades estrictamente por capas.                              | Se reducen los cambios directos a clases base usando <i>Adapter</i> y <i>Decorator</i> . | Se facilita el cambio dinámico de algoritmos de búsqueda con <i>Strategy</i> y la gestión pasiva de logs con <i>Observer</i> . | Justificación y reflexión técnica.   |
| <b>Diseño Orientado a Objetos</b>         | Se identifican clases base, métodos puros y responsabilidades del negocio.             | Se aplican interfaces estructurales aisladas.                                            | Se inyectan dependencias de comportamiento dinámico.                                                                           | Diagramas de componentes unificados. |

### 3.3.5. Evidencias de ejecución y reflexión técnica

- El uso de Observer resolvió el problema del alto acoplamiento entre las operaciones nativas del CRUD y los sistemas accesorios de monitoreo. Si hubiésemos integrado la lógica de guardar historiales y emitir alertas directamente en los métodos de registro o actualización, la clase principal habría perdido su alta cohesión. Observer permitió que el servicio base únicamente notifique el cambio de estado, delegando la responsabilidad de procesamiento a los observadores externos.
- El uso de Strategy resolvió la rigidez de los algoritmos de filtrado. Ante la necesidad de realizar búsquedas por distintos campos (ID o Nombre), Strategy nos permitió encapsular cada lógica matemática/comparativa en su propia clase. Esto garantiza que si en el futuro se requiere buscar

por "Edad", solo se deberá agregar una nueva estrategia concreta sin tocar una sola línea del servicio principal ni de la vista.

Notificaciones dinámicas del patrón ObserverAl registrar, actualizar o eliminar un registro, el sistema reacciona automáticamente emitiendo una alerta periférica y guardando la bitácora internamente.

```
=====
CRUD DEL ESTUDIANTE - ARQUITECTURA INTEGRADA U2T3
Decorator (Validaciones Extras): DESACTIVADO
=====
1. Registrar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
----- Patrones Estructurales -----
6. [DECORATOR] Alternar Decorador de Validacion
7. [ADAPTER] Cargar datos de EstudianteExterno
----- Patrones de Comportamiento -----
8. [STRATEGY] Búsqueda Dinámica (Por ID o Nombre)
9. [OBSERVER] Ver Historial de Notificaciones
0. Salir
=====
Seleccione una opción: 1

=====
--- Registrar estudiante ---
Ingrese ID: 1
Ingrese nombres: Andres Cedeño
Ingrese edad: 21
--> [OBSERVER - AUDITORÍA EN VIVO] Se detectó una operación de [REGISTRO] sobre el estudiante ID: 1
Estudiante registrado correctamente.
=====

=====
9. [OBSERVER] Ver Historial de Notificaciones
0. Salir
=====
Seleccione una opción: 9

--- [OBSERVER] Historial de Notificaciones (Bitácora) ---
Evento: REGISTRO | Estudiante: Andres Cedeño (ID: 1)
Evento: ACTUALIZACIÓN | Estudiante: Andres Isaiás Cedeño (ID: 1)
=====

=====
Seleccione una opción: 4

--- Actualizar estudiante ---
Ingreso ID del estudiante a actualizar: 1
Ingreso nuevos nombres: Andres Isaiás Cedeño
Ingreso nueva edad: 21
--> [OBSERVER - AUDITORÍA EN VIVO] Se detectó una operación de [ACTUALIZACIÓN] sobre el estudiante ID: 1
Estudiante actualizado correctamente.
=====
```

Cambio de algoritmo dinámico con el patrón Strategy. El sistema permite permutar entre la búsqueda numérica (ID) y la búsqueda de aproximación textual (Nombres) de forma transparente.

```
=====
CRUD DEL ESTUDIANTE - ARQUITECTURA INTEGRADA U2T3
Decorator (Validaciones Extras): DESACTIVADO
=====
1. Registrar estudiante
2. Listar estudiantes
3. Consultar estudiante por ID
4. Actualizar estudiante
5. Eliminar estudiante
----- Patrones Estructurales -----
6. [DECORATOR] Alternar Decorador de Validacion
7. [ADAPTER] Cargar datos de EstudianteExterno
----- Patrones de Comportamiento -----
8. [STRATEGY] Búsqueda Dinámica (Por ID o Nombre)
9. [OBSERVER] Ver Historial de Notificaciones
0. Salir
=====
Seleccione una opción: 8

--- [STRATEGY] Búsqueda Dinámica ---
1. Buscar por ID
2. Buscar por Nombre
Seleccione estrategia: 2
Ingrese el valor a buscar: Jean

[STRATEGY] Ejecutando algoritmo de búsqueda seleccionado...
--- Resultados de Búsqueda ---
ID: 3 | Nombre: Jeancarlo Santi

--- [STRATEGY] Búsqueda Dinámica ---
1. Buscar por ID
2. Buscar por Nombre
Seleccione estrategia: 1
Ingrese el valor a buscar: 2

[STRATEGY] Ejecutando algoritmo de búsqueda seleccionado...
--- Resultados de Búsqueda ---
ID: 2 | Nombre: Erick Tufiño
```

### **3.4. Conclusión**

Como grupo creemos que la aplicación conjunta de los patrones Observer y Strategy ha dotado a nuestro sistema de un nivel de flexibilidad excepcional. Pensamos que la implementación de Observer fue clave para mantener limpio el núcleo transaccional, permitiendo agregar sistemas de monitoreo y bitácoras sin ensuciar los métodos del CRUD original. De igual forma, consideramos que Strategy simplificó drásticamente la capa de presentación y de negocio, eliminando los clásicos bloques condicionales extensos al delegar la responsabilidad de la búsqueda a algoritmos encapsulados intercambiables. Todo esto demuestra en la práctica cómo el comportamiento de un software puede alterarse radicalmente en tiempo de ejecución de forma segura.

### **3.5. Recomendación**

Recomendamos que, para asegurar la escalabilidad futura del módulo de reportes, se persistan los eventos recolectados por el HistorialObserver en una base de datos o archivo físico al finalizar la ejecución del sistema. Asimismo, sugerimos expandir las interfaces del patrón Strategy para abarcar algoritmos de ordenamiento (ej. Ordenar por Edad Ascendente/Descendente), lo cual maximizará la reutilización de código en la capa de presentación cuando el CRUD deba renderizar tablas de datos masivas.